

LAPORAN INDIVIDU
UJIAN TENGAH SEMESTER (UTS)
PRAKTIKUM COMMUNICATION PROTOCOL
STUDI KASUS: CASE 2 - PRODUCT/INVENTORY API

Mata Kuliah : Communication Protocols



Disusun Oleh:

ADITYA WARDANA
(25120500001)

[Link Repository GitHub](#)

FAKULTAS ILMU KOMPUTER
PROGRAM STUDI SAINS DATA
UNIVERSITAS CAKRAWALA
TAHUN 2026

DAFTAR ISI

COVER.....	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iii
BAB I RINGKASAN CASE	1
A. Identitas Case	1
B. Tujuan API dan Protocol.....	1
C. Skenario Bisnis dan Data	1
BAB II DESAIN REQUEST	3
A. Request 1: Mengambil Daftar Produk (GET All)	3
B. Request 2: Mengambil Detail Spesifik Produk (GET by ID)	3
C. Request 3: Menambahkan Produk Inventaris Pertama (POST).....	3
D. Request 4: Menambahkan Produk Inventaris Kedua (POST)	4
BAB III EVIDENCE GET.....	5
A. Evidence GET Daftar Produk	5
B. Evidence GET Detail Produk.....	6
BAB IV EVIDENCE POST	7
A. Evidence POST Tambah Produk Pertama	7
B. Evidence POST Tambah Produk Kedua	8
BAB V ANALISIS RESPONSE.....	9
A. Analisis Status Code	9
B. Analisis HTTP Headers	9
C. Analisis Struktur Body dan Tipe Data	9
D. Analisis Error Handling dan Troubleshooting	10
BAB VI ANALISIS ENCODING DATA	11
A. Format dan Standar Encoding.....	11
B. Struktur Sintaksis (Key-Value Pairs)	11
C. Serialisasi dan Deserialisasi Data	11
D. Keunggulan Implementasi	12
BAB VII KESIMPULAN	13
A. Temuan Utama	13
B. Kendala dan Solusi	13
C. Pembelajaran Protokol	13

DAFTAR GAMBAR

BAB I

RINGKASAN CASE

A. Identitas Case

Praktikum Ujian Tengah Semester (UTS) ini menggunakan **Case 2 - Product/Inventory API** sebagai subjek analisis. Pemilihan studi kasus ini sejalan dengan parameter ujian yang mensyaratkan *endpoint* harus cukup jelas untuk diuji melalui alur *request-response* dan dianalisis format datanya.

B. Tujuan API dan Protocol

Tujuan utama dari pengujian *Communication Protocol* ini adalah:

- Memverifikasi keberhasilan komunikasi *client-server* melalui terminal *command line* menggunakan utilitas *curl*.
- Melakukan observasi terhadap lalu lintas *request* dan *response* HTTP, dengan fokus pada pengamatan *status code*, kelengkapan *headers*, serta struktur *body*.
- Menganalisis format *encoding* data yang digunakan dalam transmisi informasi, yang dalam kasus ini direpresentasikan dengan format JSON (*JavaScript Object Notation*).

C. Skenario Bisnis dan Data

Skenario data pada pengujian ini menyimulasikan sistem *backend* dari manajemen inventaris produk atau barang. Sesuai dengan spesifikasi praktikum yang mewajibkan minimal 4 eksekusi *request*, skenario dipecah menjadi dua interaksi protokol:

1. Pengambilan Data (*Data Retrieval*):

Skenario ini menggunakan HTTP *method* GET untuk memanggil *resource* dari *server*. Pengujian mencakup penarikan rekam jejak keseluruhan daftar produk dari basis data (*GET daftar produk*) dan penarikan informasi spesifik untuk satu entitas produk berdasarkan identitas uniknya (*GET detail produk*).

2. Penulisan Data (*Data Submission*):

Skenario ini menggunakan HTTP *method* POST untuk mengirimkan *payload* data ke *server*. *Client* mensimulasikan penambahan produk baru ke dalam inventaris (*POST*

tambah produk) dengan melampirkan *body request* terencode JSON yang memuat atribut produk secara terstruktur.

BAB II

DESAIN REQUEST

Desain pengujian ini memetakan empat interaksi spesifik pada *endpoint* api/products yang terbagi ke dalam dua metode HTTP utama (GET dan POST). Berikut adalah spesifikasi teknis dari masing-masing *request*:

A. Request 1: Mengambil Daftar Produk (GET All)

Method	GET
Endpoint	http://localhost:8088/api/products
Headers	Tidak memerlukan transmisi <i>header</i> khusus dari sisi <i>client</i> .
Body	Kosong (Metode GET tidak mendefinisikan pengiriman <i>payload</i>).
Expected Response	<i>Server</i> diharapkan merespons dengan <i>Status Code</i> 200 OK dan mengembalikan <i>body response</i> berformat JSON. Struktur data yang diharapkan adalah sebuah <i>array of objects</i> yang memuat rincian inventaris secara komprehensif.

B. Request 2: Mengambil Detail Spesifik Produk (GET by ID)

Method	GET
Endpoint	http://localhost:8088/api/products/1 (Memanggil <i>resource</i> dengan ID 1).
Headers	Tidak memerlukan transmisi <i>header</i> khusus.
Body	Kosong.
Expected Response	<i>Server</i> diharapkan merespons dengan <i>Status Code</i> 200 OK. <i>Body response</i> harus berupa sebuah <i>object</i> JSON tunggal yang memuat struktur data spesifik dari produk ID 1.

C. Request 3: Menambahkan Produk Inventaris Pertama (POST)

Method	POST
Endpoint	http://localhost:8088/api/products
Headers	Wajib menyertakan Content-Type: application/json. Parameter ini menginstruksikan <i>server</i> bahwa <i>payload</i> yang dikirimkan terencode dalam format JSON.
Body Request	JSON <pre>{ "name": "Smart Sensor", "category": "iot", "price": 175000, "stock": 12 }</pre>
Expected Response	<i>Server</i> akan memvalidasi <i>body request</i> . Jika berhasil, <i>server</i> harus merespons dengan <i>Status Code</i> 201 Created yang menandakan

	<i>resource</i> baru berhasil dialokasikan di dalam basis data. <i>Body response</i> diharapkan menampilkan kembali objek data yang dikirim beserta id unik dan <i>timestamp</i> pembuatan.
--	---

D. Request 4: Menambahkan Produk Inventaris Kedua (POST)

Method	POST
Endpoint	http://localhost:8088/api/products
Headers	Content-Type: application/json
Body Request	JSON <pre>{ "name": "Microcontroller NodeMCU", "category": "iot", "price": 65000, "stock": 25 }</pre>
Expected Response	Serupa dengan Request 3, hasil yang diharapkan adalah <i>Status Code</i> 201 Created yang dikonfirmasi dengan <i>output</i> JSON berisi data "Microcontroller NodeMCU" dengan indeks ID terbarunya.

BAB III

EVIDENCE GET

A. Evidence GET Daftar Produk



```
PS C:\Users\warda> curl.exe -i -X GET "http://localhost:8088/api/products"
HTTP/1.0 200 OK
Server: CommProtocolsDemo/1.0 Python/3.13.3
Date: Sun, 07 Jun 2026 12:36:07 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 642
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type

{
  "resource": "products",
  "case": "Product",
  "count": 3,
  "methods": [
    "GET",
    "POST",
    "PUT",
    "PATCH",
    "DELETE"
  ],
  "canonicalPath": "/api/products",
  "itemPathExample": "/api/products/1",
  "data": [
    {
      "id": 1,
      "name": "Laptop Pro 14",
      "category": "computer",
      "price": 14500000,
      "stock": 5
    },
    {
      "id": 2,
      "name": "Temperature Sensor DHT22",
      "category": "iot",
      "price": 85000,
      "stock": 30
    },
    {
      "id": 3,
      "name": "Wireless Router AC1200",
      "category": "networking",
      "price": 475000,
      "stock": 10
    }
  ]
}
```

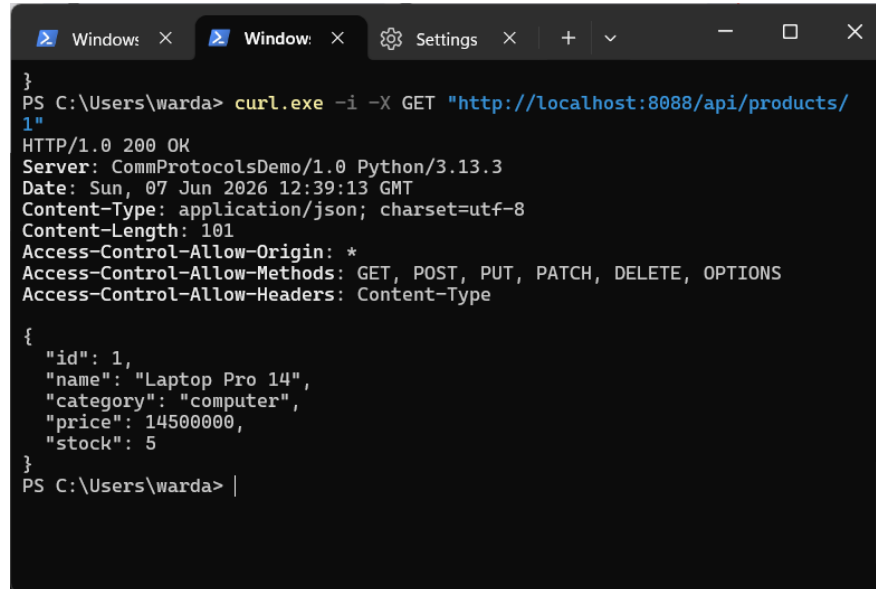
Eksekusi pertama menggunakan *command*

```
curl.exe -i -X GET "http://localhost:8088/api/products"
```

berjalan dengan sukses. Parameter `-i` berfungsi untuk menampilkan *HTTP headers* dari *server*. Berdasarkan tangkapan layar, *server* merespons dengan status code `HTTP/1.0 200 OK`, yang mengindikasikan bahwa *request* telah diterima dan diproses tanpa kesalahan. *Header Content-Type: application/json; charset=utf-8* memastikan data dikembalikan dalam format JSON. *Body response* menampilkan sebuah objek yang berisi *array* "data", yang mendata tiga produk

awal "Laptop Pro 14", "Temperature Sensor DHT22", dan "Wireless Router AC1200") secara terstruktur.

B. Evidence GET Detail Produk



```
PS C:\Users\warda> curl.exe -i -X GET "http://localhost:8088/api/products/1"
HTTP/1.0 200 OK
Server: CommProtocolsDemo/1.0 Python/3.13.3
Date: Sun, 07 Jun 2026 12:39:13 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 101
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type

{
  "id": 1,
  "name": "Laptop Pro 14",
  "category": "computer",
  "price": 14500000,
  "stock": 5
}
```

Eksekusi kedua difokuskan pada pemanggilan spesifik menggunakan *command*

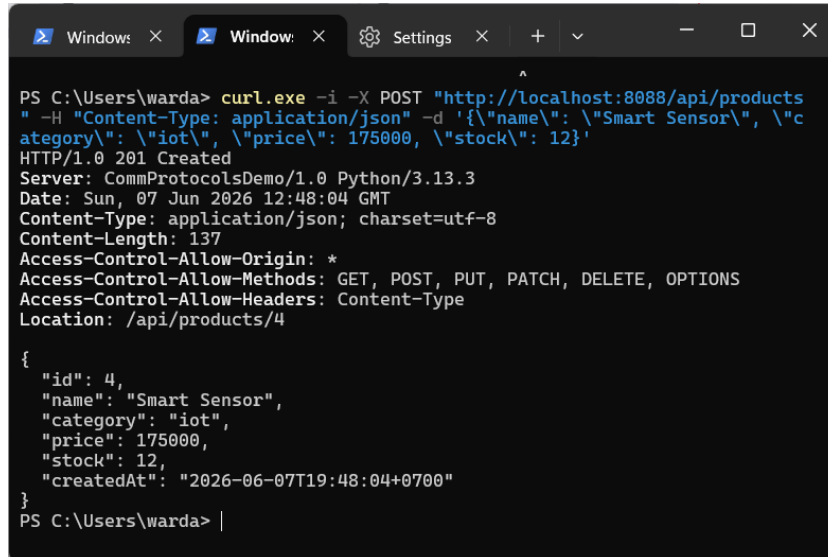
```
curl.exe -i -X GET "http://localhost:8088/api/products/1"
```

Server secara valid mengidentifikasi ID 1 dan mengembalikan status code `HTTP/1.0 200 OK`. Berbeda dengan *request* pertama yang mengembalikan struktur *array of objects*, *request* ini secara presisi hanya mengembalikan satu objek JSON (*single object*). Data yang ditarik adalah "Laptop Pro 14" dengan tipe kategori "computer", harga 14500000, dan stok 5.

BAB IV

EVIDENCE POST

A. Evidence POST Tambah Produk Pertama



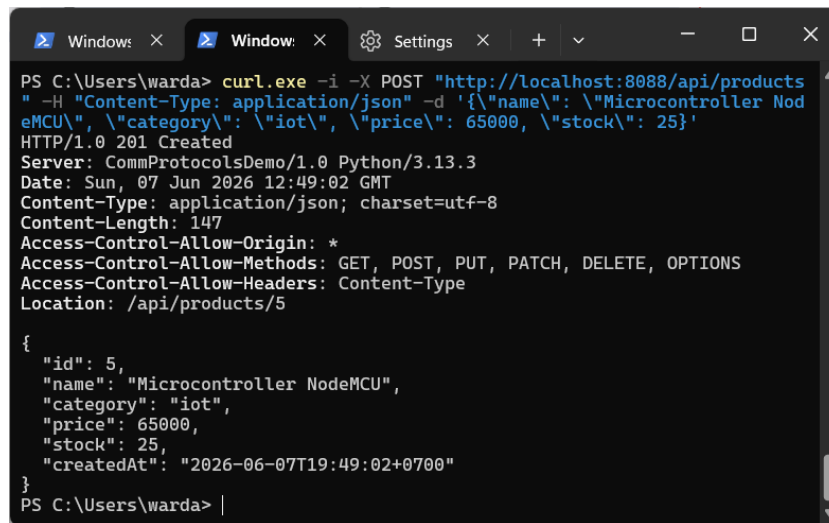
```
PS C:\Users\warda> curl.exe -i -X POST "http://localhost:8088/api/products"
-H "Content-Type: application/json" -d '{"name": "Smart Sensor", "category": "iot", "price": 175000, "stock": 12}'
HTTP/1.0 201 Created
Server: CommProtocolsDemo/1.0 Python/3.13.3
Date: Sun, 07 Jun 2026 12:48:04 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 137
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type
Location: /api/products/4

{
  "id": 4,
  "name": "Smart Sensor",
  "category": "iot",
  "price": 175000,
  "stock": 12,
  "createdAt": "2026-06-07T19:48:04+0700"
}
PS C:\Users\warda> |
```

Pengujian metode POST dilakukan dengan menginjeksi *payload* JSON baru ke *server*. *Header* wajib

`-H "Content-Type: application/json"` ditambahkan agar *server* tidak menolak format teks yang dikirimkan. Eksekusi penambahan produk "Smart Sensor" berhasil diotorisasi oleh *server*, dibuktikan dengan kemunculan status code `HTTP/1.0 201 Created`. *Status code* 201 adalah standar HTTP yang menegaskan bahwa *resource* baru telah berhasil diciptakan di basis data. *Body response* mengonfirmasi hal ini dengan mencetak kembali *payload* yang dikirimkan beserta penambahan *field* baru yang di-generate otomatis oleh *server*, yaitu "id": 4 dan *timestamp* "createdAt".

B. Evidence POST Tambah Produk Kedua



```
PS C:\Users\warda> curl.exe -i -X POST "http://localhost:8088/api/products" -H "Content-Type: application/json" -d '{"name": "Microcontroller NodeMCU", "category": "iot", "price": 65000, "stock": 25}'
HTTP/1.0 201 Created
Server: CommProtocolsDemo/1.0 Python/3.13.3
Date: Sun, 07 Jun 2026 12:49:02 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 147
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type
Location: /api/products/5

{
  "id": 5,
  "name": "Microcontroller NodeMCU",
  "category": "iot",
  "price": 65000,
  "stock": 25,
  "createdAt": "2026-06-07T19:49:02+0700"
}
PS C:\Users\warda> |
```

Siklus pengujian terakhir menduplikasi parameter struktur pada pengujian 5.1, namun dengan memodifikasi *value* pada *body request* menjadi produk "Microcontroller NodeMCU" dengan kategori "iot", harga 65000, dan stok 25. Eksekusi dieksekusi menggunakan utilitas curl.exe pada PowerShell dengan metode `escape string (\")` untuk menghindari pemotongan argumen oleh terminal. *Server* secara konsisten merespons dengan status `HTTP/1.0 201 Created` serta mengembalikan representasi objek dengan indeks iterasi terbaru, yaitu `"id": 5`. Eksekusi ini mengonfirmasi bahwa *endpoint* dapat menerima operasi penulisan data secara sekuensial tanpa anomali struktural.

BAB V

ANALISIS RESPONSE

Berdasarkan eksekusi *request* yang telah dilakukan, berikut adalah analisis mendetail terhadap komponen respons dari *server*:

A. Analisis Status Code

- **200 OK**
Diterima pada eksekusi metode GET. Kode ini menandakan bahwa *request* berhasil diproses dan *server* mengembalikan informasi *resource* yang diminta secara valid.
- **201 Created**
Diterima pada eksekusi metode POST. Kode ini secara spesifik mengonfirmasi bahwa *payload* data berhasil diterima, divalidasi, dan *resource* baru telah berhasil diinjeksi ke dalam basis data (ditandai dengan *generate* nilai id baru).

B. Analisis HTTP Headers

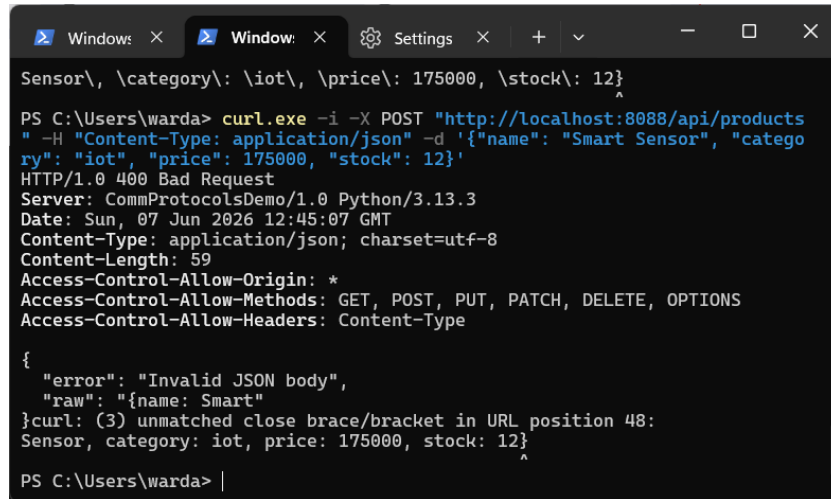
- **Content-Type: application/json; charset=utf-8**
Mendefinisikan bahwa transmisi data dari *server* dibentuk dalam format JSON, didukung dengan standar *encoding* karakter UTF-8.
- **Content-Length**
Menginformasikan ukuran *body response* (dalam *bytes*), memungkinkan sisi *client* untuk mengestimasi alokasi memori yang diperlukan saat membaca respons.
- **Access-Control-Allow-Methods**
Menampilkan daftar metode operasional yang diizinkan oleh sistem (GET, POST, PUT, PATCH, DELETE, OPTIONS), merepresentasikan adanya kebijakan CORS pada *server*.

C. Analisis Struktur Body dan Tipe Data

- **Struktur Objek**
Pada pemanggilan keseluruhan inventaris (*/api/products*), *server* membungkus data dalam *array of objects*. Pada pemanggilan spesifik produk (*/api/products/1*), data direpresentasikan langsung sebagai *single object*.
- **Tipe Data Primer**
Atribut deskriptif (*name*, *category*, *createdAt*) diformat sebagai tipe data *String*. Atribut

yang memuat nilai hitung atau identifikasi (id, price, stock) diformat secara ketat sebagai tipe data *Integer*.

D. Analisis Error Handling dan Troubleshooting



```
Sensor\, \category\:\iot\, \price\: 175000, \stock\: 12}
PS C:\Users\warda> curl.exe -i -X POST "http://localhost:8088/api/products" -H "Content-Type: application/json" -d '{"name": "Smart Sensor", "category": "iot", "price": 175000, "stock": 12}'
HTTP/1.0 400 Bad Request
Server: CommProtocolsDemo/1.0 Python/3.13.3
Date: Sun, 07 Jun 2026 12:45:07 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 59
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS
Access-Control-Allow-Headers: Content-Type

{
  "error": "Invalid JSON body",
  "raw": "{name: Smart
}curl: (3) unmatched close brace/bracket in URL position 48:
Sensor, category: iot, price: 175000, stock: 12}
PS C:\Users\warda> |
```

Sistem memiliki mekanisme penanganan anomali (*error handling*) yang responsif. Ketika terjadi percobaan eksekusi POST tanpa format *string* yang sesuai di PowerShell, sistem mendeteksi cacat pada struktur data dan menolak transmisi dengan status `HTTP/1.0 400 Bad Request`.

Body response tidak menampilkan pesan generik, melainkan secara spesifik merinci akar kendala melalui paramater `"error": "Invalid JSON body"` dan menangkap potongan teks yang terputus `"raw": "{name: Smart"`. Resolusi teknis yang diambil untuk menangani kendala lingkungan eksekusi terminal Windows ini adalah mengimplementasikan karakter pelolosan (*escape string* menggunakan `\`) pada setiap *key* dan *value* di *payload*, memastikan JSON dapat diproses utuh oleh *server*.

BAB VI

ANALISIS ENCODING DATA

Sesuai dengan spesifikasi RESTful API pada skenario pengujian ini, pertukaran data antara *client* (utilitas CLI) dan *server* tidak menggunakan XML atau Protobuf, melainkan murni menggunakan representasi **JSON** (*JavaScript Object Notation*). Berikut adalah analisis struktural dari format *encoding* tersebut:

A. Format dan Standar Encoding

Seluruh *traffic* data pada eksekusi GET maupun POST diencode ke dalam format JSON dengan karakter *encoding* berbasis **UTF-8**. Hal ini dikonfirmasi oleh *header* `Content-Type: application/json; charset=utf-8` yang mengikat standar komunikasi agar kedua belah pihak dapat membaca karakter secara seragam tanpa distorsi.

B. Struktur Sintaksis (Key-Value Pairs)

Format JSON yang ditransmisikan menggunakan arsitektur *key-value pairs* (pasangan kunci dan nilai).

- Atribut pengenalan (*key*) seperti `"name"`, `"category"`, dan `"price"` selalu diapit oleh tanda kutip ganda.
- Representasi objek individual dibatasi oleh tanda kurung kurawal `{}` (seperti pada respons GET tunggal dan *payload* POST).
- Representasi koleksi data dibatasi oleh tanda kurung siku `[]` (seperti pada respons GET `/api/products` yang memuat *array* dari beberapa objek produk).

C. Serialisasi dan Deserialisasi Data

Proses *encoding* dalam praktikum ini melibatkan dua fase krusial:

1. Serialisasi (*Serialization*)

Saat menjalankan perintah POST melalui PowerShell, *payload* data yang diinput pengguna (`{\"name\": \"Smart Sensor\"...}`) diserialisasi menjadi format *string* murni agar dapat ditransmisikan melalui protokol HTTP secara ringan.

2. Deserialisasi (*Deserialization*)

Saat *string* HTTP tersebut tiba di *server* (yang teridentifikasi menggunakan *environment* Python/3.13.3 pada *header*), sistem *server* mem-parsing (*deserialize*) *string* tersebut kembali menjadi struktur data internal yang valid sebelum disimpan ke *database*.

D. Keunggulan Implementasi

Penggunaan JSON pada *endpoint* ini terbukti efisien. Format ini sangat *lightweight* (ringan) sehingga meminimalkan beban *bandwidth*, sekaligus bersifat *human-readable* (mudah dibaca oleh manusia) yang sangat mempermudah proses observasi dan *troubleshooting* manual melalui terminal seperti yang dilakukan pada praktikum ini.

BAB VII

KESIMPULAN

A. Temuan Utama

Praktikum ini berhasil memverifikasi interaksi protokol HTTP pada *Case 2* (Product/Inventory API). Pengujian menyimpulkan bahwa *endpoint* berfungsi dengan baik sesuai standar RESTful API. Hal ini dibuktikan melalui respons *server* yang mengembalikan *status code* 200 OK secara konsisten pada operasi pengambilan data (GET) dan 201 Created pada operasi penulisan data (POST). Seluruh transmisi data dieksekusi secara efisien menggunakan format *encoding* JSON dengan standar karakter UTF-8.

B. Kendala dan Solusi

Terdapat satu kendala teknis yang ditemukan selama proses praktikum berlangsung. Saat mengeksekusi metode POST menggunakan *command line* PowerShell, *server* menolak *request* dan memberikan status HTTP 400 Bad Request. Analisis menunjukkan bahwa hal ini disebabkan oleh ketidakmampuan terminal dalam mempertahankan integritas *payload* JSON yang mengandung spasi. Masalah ini berhasil diselesaikan dengan teknik *escape string*, yaitu menambahkan karakter *backslash* (\) pada setiap *key* dan *value* di dalam format JSON, sehingga *server* dapat memproses *body request* secara utuh.

C. Pembelajaran Protokol

Melalui studi kasus ini, dapat dipahami bahwa keberhasilan dan integritas komunikasi *client-server* sangat bergantung pada akurasi parameter HTTP. Penyertaan *header* yang tepat, seperti Content-Type: application/json, bersifat wajib agar *server* mengenali jenis informasi yang dikirimkan. Selain itu, praktikum ini memberikan pemahaman mendalam mengenai siklus serialisasi dan deserialisasi data, di mana utilitas pengirim (*client*) harus menyusun data secara logis sebelum ditransmisikan, dan *server* bertugas mem-parsing data tekstual tersebut kembali menjadi struktur data yang valid sebelum dimasukkan ke dalam *database*.